

Super admin — brainstorm

Owner: PA Office backend

Status: Parked reference — the ideation behind the Settings-first build now underway.

Author: Engineering

Date: 2026-07-06

Full landscape of what a super-admin surface could cover for a Minister's PA office, why the scope needs to be split by role, and the recommended order in which to ship. Kept for reference — the immediate build focuses on the Settings page (users, departments, email configs, department logins) as the seed of the role-based access system.

1. The strategic question first

Before designing screens, decide **who the super admin actually is** and what they should NOT touch. Two mental models:

- **Platform operator** — the CM's IT lead or the vendor's on-call engineer.

Sees everything, touches infrastructure, doesn't handle citizen data day to day.

- **Office administrator** — the MLA's senior aide who runs the office

platform. Manages staff, config, templates — not credentials or backups.

Both are needed because they need different guardrails.

Role	Owns
Super Admin (platform)	API keys, encryption, integrations, LLM/AI config, audit log, backups, feature flags, provisioning new offices
Office Admin (per MLA)	Staff accounts inside their office, MLA profile, meeting hours + holidays, notification template copy, SLA targets, category catalog display labels
PA (existing)	Daily case work
Dept account (existing)	Department workspace
Auditor (new, read-only)	Audit log + reports for CM's office, journalists, audit team

Without the split, "super admin" becomes the "one login that can do anything" — which is what we have today with admin/admin123. That is the political liability shape.

2. Eight admin surfaces

Grouped by concern, ordered by priority.

2.1 Users & sessions (ship first)

Solves the current bearer-key problem.

- Invite / list / disable staff accounts (email + name + role)
- Per-user audit trail (last-login, IP, actions)
- Force sign-out (invalidate all sessions for a user)
- Reset department account passwords (this was the blocker earlier)
- Rotate floor-display credentials
- Bootstrap: first super admin still comes from env, invites everyone else

2.2 Office & MLA setup

Currently hardcoded: 08:00–18:00, 30-min slots, 12 per slot, single MLA.

- MLA profile (name, constituency, contact, portrait, active/inactive)
- Meeting hours per weekday (Mon–Sat may differ)
- Slot duration, max citizens per slot
- Working days + calendar of holidays / block-out dates
- Auto-reschedule policy time and rule

2.3 Catalog management

Ministries (34), categories (13), sub-departments (10) are Python enums today. Recommended split:

- **Values** stay as enums — compile-time safe, referenced in code
- **Display labels** (English + Tamil) live in a `catalog_labels` DB table so

admin can rename without a deploy

- **Deprecation** — mark a value inactive so new petitions cannot be

classified under it, historical data still renders

Full DB-driven enums would let admin add a whole new ministry from the UI, but then Gemini's response schema drifts at runtime. Not worth it yet.

2.4 AI / LLM configuration

Ties into the parked LLM platform migration plan.

- Provider toggle: Google AI Studio ↔ Vertex AI
- Model per surface (summarisation, AI Uploads, STT)
- Fallback chain
- Priority tier (priority / standard / flex)
- **Prompt version switching** — critical, lets us roll back a bad prompt

without a deploy

- API key / GCP-project rotation with masked display + audit row on change

2.5 Notification templates

Bilingual, one per event: APPOINTMENT_SCHEDULED, RESCHEDULED, TICKET_RESOLVED, MEETING_REMINDER_TOMORROW, etc.

- Variables: `{{token}}`, `{{citizen_name}}`, `{{time}}`, `{{ministry}}`
- Optional approval workflow before edited templates go live
- Channel: SMS / WhatsApp / both
- Preview + send-test-to-my-number

2.6 Ticket workflow tuning

- SLA targets per priority (currently hardcoded: critical=3d, high=7d, medium=14d, low=28d)
- Closure reason list
- Auto-close policy for stale tickets

2.7 Audit log viewer

Valuable once the v2 activity table has real volume.

- Filter by user / action_type / date range / entity
- Export to CSV for compliance queries
- Diff view for config changes (before/after JSON)
- Retention policy display

2.8 Data governance

- Retention: auto-archive appointments after N years
- Citizen deletion request workflow (soft-delete with reason)
- Encryption key rotation (two-person rule candidate)
- Backup status (read-only view of last DB snapshot)

3. Data model — how to store it

Hybrid approach:

- **Per-domain tables** for structured concerns: `sla_targets`, `notification_templates`, `catalog_labels`, `office_settings`, `users`. Each table gets proper foreign keys and constraints.
- **Single** `feature_flags` **table** for boolean toggles.
- **Single** `integrations` **table** (encrypted) for API keys.

Every config write becomes one activity row (`action_type = 'CONFIG_CHANGED'`, `payload = {domain, field, from, to}`) — free audit trail using the v2 activity model.

4. Open questions worth answering upfront

1. **Bootstrap the first super admin** — env var still needed for user #1

(same as today), then they invite others. Or provision via CLI. Which do we prefer when someone gets locked out?

2. **Where do secrets live?** — three choices in ascending safety:

- Fernet-encrypted with the same key we use for PII (fast, but one key

controls everything)

- Fernet-encrypted with a separate key stored in env (cheap, adds one env

var)

- GCP Secret Manager (proper KMS, needs the Vertex migration to be done

first)

3. **Do config changes need approval?** — for SMS templates: probably YES

(two-person rule so no rogue actor edits an SMS to say something political). For SLA changes: probably NO.

4. **How real-time?** — some configs (SLA targets, template text) safe to

hot-reload every N seconds. Others (LLM provider switch, prompt version) need a graceful drain of in-flight requests.

5. **Multi-office future** — if a second MLA/office is real 2027 target, the

schema should be tenanted **now** with an `office_id` on the appropriate tables. Retrofit later is painful.

6. **Feature-flag first** — since several parked plans exist (token dedup,

Vertex migration, own LLM), a feature-flags surface pays for itself immediately.

5. Recommended build order

Small first pass, high value:

1. **Users & sessions** — solves a real security issue today
2. **Feature flags** — unlocks the parked plans without deploys
3. **Prompt versioning + LLM config** — lets us iterate on prompts safely in

prod

4. **Catalog labels** — cheap, obvious, useful

Then, based on demand:

5. Notification templates (once we're sending SMS at volume)
 6. Office / MLA setup (once we onboard a second MLA)
 7. Ticket workflow tuning (once SLA misses become a real signal)
 8. Audit log viewer (once activity table data is worth mining)
-

6. Chosen direction (current build)

Settings-first, users + departments + email configs — the user's requested scope. Roles are the seed of the future RBAC:

- Create PA / super-admin users (with roles)
- Add extra school sub-departments beyond the seeded 10
- Configure email address for each sub-department (for the auto-forward

workflow)

- Configure email address for each Ministry (for the escalation workflow)
- Create login accounts for department staff

Implementation notes in the companion `settings-implementation-plan.md` (pending).